

By Md. Samiul Islam Nabil

Index

Topic	Page
Mechanism Inside Print Function	1
Data Types	1
Comment	1
Variable	1
Keywords	2
Identifiers	2
Input Function	2
Type Conversion	3
Literals	3
Operators	5
If , Else , Else-if Condition	6
Indentation	6
While Loop	7
Range	8
Sequence	8
For Loop	9
Nested Loop	9
Break , Continue , Pass	10
Functions	10
Modules	12
Accessing Substrings from a String & Indexing and Slicing Strings	13
String Operations in Python:	14
String Functions	16

Mechanism Inside Print Function

```
print("what I want to write")
print("nabil", "1", "true") => for printing different types of data type
print(" ", sep= 'anything I want to print between two words/ strings')
print(" ", end= " ") => in default the value is \n but we can set anything between two lines
point : " " or ' ' define same thing on python . and in one print " ", " " define as different string.
```

Data Types

- Basic Types - (integer, float, complex, boolean and string)

Ex: print(any num)

Ex: print(true/false)

Ex: print (1+2j)

Point: the highest num in integer can print is 1e308

- Container Types - (List, Tuples, Sets and Dictionary)

1 print([1,2,3,4,5]) => list

2 print((1,2,3,4,5)) => tuple

3 print({1,2,3,4}) => set

4 print({"name": "nabil", "age": 21, "gender": "male"}) => dictionary

- User-defined Types – Class

Comment

For using comment use # . and for multiline """ codes """

Variable

Concept of Dynamic and Static data typing : dynamic data type means there is no need to define data type (eg..python, php). But in static there have to define data type (eg.. c,c++,java)

Ex: name = 'any data type' => for using as string

```
print(name)
```

result: the value stored

Concept of Dynamic and Static Binding: In Dynamic binding we can change into any data type on program life time (eg..python). But in static once the variable is declared is fixed (eg.. c,c++,java)

Ex: name = anything without character (because character goes for string)

```
print(name)
```

result: the value stored

Special Syntax:

```
a = 5;b=6;c=7
```

```
print(a)
```

```
print(b)
print(c)
a,b,c = 4,5,6
print(a)
print(b)
print(c)
```

```
a=b=c=6
print(a)
print(b)
print(c)
```

Keywords

Python has 33 key words.

The code for showing keyword: import keyword
 print(keyword.kwlist)

they are: ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

Identifiers

Rules for setting Identifiers

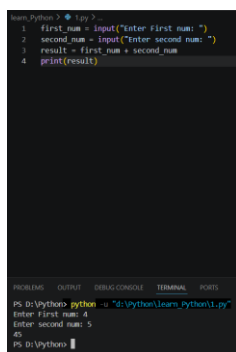
- can only start with an alphabet or _
- Followed by 0 or more letter, _ and digits
- keywords cannot be used as an identifiers

Input Function

For taking input we have to use input() .

Ex: input("what is your name: ")

Point: python always takes input in string format. Because string is universal format.



```

1: first_num = input("Enter first num: ")
2: second_num = input("Enter second num: ")
3: result = first_num + second_num
4: print(result)

PS D:\Python> python -u "d:\python\learn_python1.py"
Enter first num: 4
Enter second num: 5
45
PS D:\Python>

```

Here is the example. 5+6 = is not 11 ..here the numbers added as string ..as a result it prints 56.

For knowing data type: print(type(anything))

Type Conversion:

Mainly There have two types of type conversion. They are implicit (where the programmer no need to define anything interpreter automaticly converts.) and Explicit (where the programmer need to define).

Implicit: ex: `print(5+6.9) = 11.9`

Explicit: ex:

```
first_num = int(input("Enter First num: "))
second_num = int(input("Enter second num: "))
result = first_num + second_num
print(result)
```

Literals

Literal is a raw data given in a variable. In Python, there are various types of literals they are as follows:

- Numeric Literals
- String Literals
- Boolean Literals
- Special Literals

1. Numeric Literals

Input:

```
a = 0b1010 #Binary Literals
b = 100 #Decimal Literal
c = 0o310 #Octal Literal
d = 0x12c #Hexadecimal Literal
```

```
#Float Literal
float_1 = 10.5
float_2 = 1.5e2
float_3 = 1.5e-3
```

```
#Complex Literal
x = 3.14j
```

```
print(a, b, c, d)
print(float_1, float_2, float_3)
print(x, x.imag, x.real)
```

Output:

```
10 100 200 300
10.5 150.0 0.0015
3.14j 3.14 0.0
```

2. String Literals

Input:

```

string = 'This is Python'
strings = "This is Python"
char = "C"
multiline_str = """This is a multiline string with more than one line code."""
unicode = u"\U0001f600\U0001F606\U0001F923"
raw_str = r"raw \n string"

```

```

print(string)
print(strings)
print(char)
print(multiline_str)
print(unicode)
print(raw_str)

```

output:

```

This is Python
This is Python
C
This is a multiline string with more than one line code.
😄😄🤖
raw \n string

```

3. Boolean Literal

Input:

```

a = True + 4
b = False + 10

```

```

print("a:", a)
print("b:", b)

```

output:

```

a: 5
b: 10

```

4. Special Literal

Input

```

a = None
print(a)

```

output:

```

None

```

Operators

Operators are used to perform operations on variables and values. Python has the following operators:

- Arithmetic operators
- Comparison operators
- Logical operators
- Bitwise operators
- Assignment operators
- Identity operators
- Membership operators

Arithmetic operators

`print(x ** y) = x^y`

`print(x // 2) = ans of x/y without fraction number`

Comparison operators

`print(x>y)` also `<`, `=`, `!=`

Logical operators

Use of `and`, `or`, `not`

Ex: Input: `x = True y = False`

`print(x or y)`

`print(x and y)`

`print(not y)`

Output: True

False

True

Bitwise operators

Left shift (`<<`) <https://www.youtube.com/watch?v=RDSuXwsECc> /2^n

Right Shift (`>>`) <https://www.youtube.com/watch?v=uuAeBFYjPIo> *2^n

1s compliment (`~`)

Also use of `&`, `|` ex: `(x&y)`

Assignment operators

For assuming

Ex: input: `a=3`

`a += 3`

`print(a)`

`a&=6` (result will be bitwise and of 3 and 6)

Output: 3

In python `++`, `--` doesn't work like c and C++

Identity operators

Use of is, in, not

Ex: input: a = 3 x = "Delhi"
 b = 3 print("D" not in x)
 print(a is b)

Output: True False

If , Else , Else-if Condition

In python we have to use elif instead of else if .

```
Ex: input:
email = input("Apna email bata")
if '@' in email:
    password = input("Apna passowrd bhi bata")

    if email == "campusx@gmail.com" and password == "1234":
        print("Welcome")
    elif email == "campusx@gmail.com" and password != "1234":
        print("Password Incorrect")
        password = input("Password fir se bol")
        if password == "1234":
            print("Finally correct")
        else:
            print("Still incorrect")
    else:
        print("Incorrect credentials")
else:
    print("Email galat hai sahi likho")
```

Indentation:

In python semicolon ; and brackets {} don't used. Instead of this python use indentation. Indentation means space .

Ex:

```
if word in keyword.kwlist :
    print("this is keyword")
else:
    print("this is not a keyword")
```

here the space gap between if and print or else and print is indentation.

Loops:

In python there are two types of loops they are while and for loops.

While loop: Basic Format: while condition :
Statement

Ex:

```
number = int(input("Enter the number: "))

i=1

while i<=11:
    print (number , "*" , i , "=" ,number*i)
    i+=1
```

Enter the number: 24

```
24 * 1 = 24
24 * 2 = 48
24 * 3 = 72
24 * 4 = 96
24 * 5 = 120
24 * 6 = 144
24 * 7 = 168
24 * 8 = 192
24 * 9 = 216
24 * 10 = 240
24 * 11 = 264
```

A short game by while loop:

```
import random
jackpot = random.randint(1,100)

guess = int(input("Guess a number: "))

while guess != jackpot:
    if guess < jackpot:
        print("guess higher")
    else:
        print("guess lower")
    guess = int(input("Guess a number: "))

print("right number")
```

Guess a number: 56
guess higher
Guess a number: 67
guess higher
Guess a number: 77
guess higher
Guess a number: 88
guess higher
Guess a number: 90
guess higher
Guess a number: 100
guess lower
Guess a number: 94
guess higher
Guess a number: 97
guess higher
Guess a number: 99
guess lower
Guess a number: 98
right number

Range

About range : `range(x,y,z)` => x is initial value and included . y is last value but it is excluded. And z is step (the difference between elements).

Ex:

```
print(list(range(2,9,2)))
```

[2, 4, 6, 8]

By default initial value is 0 and step = 1

```
print(list(range(5)))
```

[0, 1, 2, 3, 4]

Sequence

Every element in string prints according to sequence. Like if I want to print a string Bangladesh . it will print b a n g l a d e s h . in a sequence. Then it binds together

```
print("Bangladesh")
```

Bangladesh

For loop

For loop in python mainly runs with the concept of Range and Sequence

Ex:

```
for i in range(1,11,2) :  
    print(i)
```

1
3
5
7
9

Ex:

```
for i in "bangladesh" :  
    print(i)
```

b
a
n
g
l
a
d
e
s
h

Nested loop

Loop inside loop

EX:

```
rows = int(input("Enter the number of row: "))  
  
for i in range(1, rows+1) :  
    for j in range(0,i):  
        print("*", end=" ")  
    print("")
```

Enter the number of row: 5

*
* *
* * *
* * * *
* * * * *

Break , Continue , Pass

Break: When the condition meets it breaks the loop.

```
for i in range(1,10):
    if i == 5:
        break
    print(i)
```

1
2
3
4

Continue: When the condition meets it skips the elements of condition then continue

```
for i in range(1,10):
    if i == 5:
        continue
    print(i)
```

1
2
3
4
6
7
8
9

Pass: When we have to skip any logic or function . We use pass statement .It stop the Interpreter on that statement and forward to next lines.

```
for i in range(1,10):
    pass
print("hellow nabil")
```

hellow nabil

here, the interpreter pass the logic with any error.

Fucntions:

abs(absolute) function:

```
print(abs(-4))
```

4

pow(power) Fuction:

```
print(pow(2,3))
```

8

min/max(minimum/maximum) Function:

```
print(min([1,2,7,9]))
print(max([1,2,7,9]))
print(max("bangladesh"))
```

1
9
S

Round Function:

```
c = 3.1416239392399
print(round(c,2))
```

3.14

divmod(division modulus) : prints result of x//y and x%y

```
print(divmod(5,2))
(2, 1)
```

Bin/oct/hex(binary octal hexadecimal): converts decimal into binary octal hexadecimal

```
print(bin(13))
print(oct(13))
print(hex(13))
```

0b1101
0o15
0xd

id: Shows address in memory

```
a = 3
print(id(a))
```

140723176785080

ord(ordinal): Returns Unicode value of element

```
print(ord('a'))
```

97

len(length): prints length of element

```
print(len('nabil'))
print(len({1,2,3,4,5,6}))
```

5
6

sum(sumation): prints summation of numbers

```
print(sum({1,2,3,4,5,6}))
```

21

help: describe works of any function

```
print(help("list"))
```

```
Help on class list in module builtins:

class list(object)
| list(iterable=(), /)
| Built-in mutable sequence.
|
| If no argument is given, the constructor creates a new empty list.
| The argument must be an iterable if specified.
|
| Methods defined here:
|
| __add__(self, value, /)
|     Return self+value.
|
| __contains__(self, key, /)
|
| -- More --
```

Modules

What are modules?

Consider a module to be the same as a code library.

A file containing a set of functions you want to include in your application.

Examples of python modules: math, random, os,time

Math:

```
import math
print(math.pi)
print(math.e)
```

3.141592653589793
2.718281828459045

Random:

```
import random
a = [1,2,3,5,6]
random.shuffle(a)
print(a)
```

```
PS D:\Python\learn_Python> python -u "d:\Python\learn_Python\while.py"
[1, 6, 2, 3, 5]
PS D:\Python\learn_Python> python -u "d:\Python\learn_Python\while.py"
[2, 6, 1, 5, 3]
PS D:\Python\learn_Python> python -u "d:\Python\learn_Python\while.py"
[1, 3, 2, 5, 6]
```

Accessing Substrings from a String | Indexing and Slicing in Python Strings

In python there have 2 types of string . Positive and Negative string.

Positive String: In this case. 1st word is 0th index of string.

```
c = 'hellow'
print(c[5])
```

w

Negative String: In this case. last word is -1 string index of string then second last is -2.

```
c = 'hellow'
print(c[-5])
```

e

Slicing:

Syntax: [starting:ending] where ending point is excluded

```
c = 'hello'
print(c[0:5])
```

hello

here.index count 0-4 and 5 excluded.

```
c = 'hello world'
print(c[4:])
```

o world => here skiped first 4 index

```
c = 'hello world'
print(c[:4])
```

hell

here printed only first 4 index

```
c = 'hello world'
print(c[:])
```

hello world

here printed the whole string

```
c = 'hello world'
print(c[0:8:3])
```

hlw

here 3 is step. For step 3 at first take the h then count 1 and 2 and remove them then when it is step 3 after h the letter l printed. Similarly taking l it count o as 1st space as 2nd and removed them then printed 3rd letter w.

```
c = 'hello world'
print(c[-5:-1:2])
```

wr

here as well as the the point -1 point is excluded.

```
c = 'hello world'
print(c[::-1])
```

dlrow olleh

with this syntax we can reverse the whole string

Editing and Deleting Strings in Python

Strings in python is immutable type. We ca not edit or delete any elemets of string

String Operations in Python:

Arethmatic(+ *):

```
print("Hellow" + "World")
```

HellowWorld

```
print("*" * 50)
```

Relational

```
print("America" < "Bangladesh")
```

True

Result is true because it follows lexicographical . means that .which word comes first in dictionary those word is lower. Here Bangladesh comes after America in dictionary this why Bangladesh is greater.

Logical

Python takes empty string ("") as false value. And something in string including (" ") space count as true

```
print("" and "world")
```

output is nothing means false

```
print("" or "world")
```

world

output is world because here "" means 0 and world means 1 . so the output is world.

```
print("hello" and "world")
```

world

here ans is world because . if we think hello as 1 and world is as another 1. For using and if the second output was 0 instead of 1 . the whole output was 0 means nothing. For this the second 1 means world string is main potagonist to make the statement true . so the result is world.

```
print("world" and "hellow")
```

hellow

here printed hellow because hellow is main priority string for make the statement true . as we have previously seen.

```
print("world" or "")
```

world

here world count as 1.

```
print("world" or "hellow")
```

world.

Here output is world because we know . in the case of or . if one condition is truth the whole statement is truth. Here the first word world count as 1 . for this we don't need to give priority for second 1 . so the statement is mainly truth for world as priority level . so output is world.

```
print( not "hello")
```

false.

Here hello counted as 1 . so the opposite is means false.

```
print( not "")
```

true

String Functions

len:

```
c = "Bangladesh"
print(len(c))
```

10

max: judge by highest ASCII value

```
c = "Bangladesh"
print(max(c))
```

s

min:

```
c = "Bangladesh"
print(min(c))
```

b

sorted: breaks all elements of string.

```
c = "Bangladesh"
print(sorted(c))
```

['B', 'a', 'a', 'd', 'e', 'g', 'h', 'l', 'n', 's']

```
c = "bangladesh"
print(sorted(c,reverse=True))
```

['s', 'n', 'l', 'h', 'g', 'e', 'd', 'b', 'a', 'a']

for parameter reverse= true it prints characters higher ASCII value wise.

len , min , max ,sorted are common functions which can be applied in other data type.

capitalize: Capitalize the first letter of string

```
print("i love bangladesh".capitalize())
```

I love bangladesh

title: Capitalize the first letter of every words in string

```
print("i love bangladesh".title())
```

I Love Bangladesh

upper: Capitalize Every Letter.

```
print("i love bangladesh".upper())
```

I LOVE BANGLADESH

lower: Print every letter in lowercase.

```
print("i love bangladesh".lower())
```

i love Bangladesh

swapcase: Swap the cases

```
print("i LoVe bAnGlaDeSh".swapcase())
```

I lOvE BaNgLAdEsH

count: counts particular string repaeating time

```
print("i am doing nothing".count("ng"))
```

2

find: finds the position of sub string

```
print("i am doing nothing".find("d"))
```

5

```
print("i am doing nothing".find("x"))
```

-1

Output is -1 because the sub string is missing in this string.

index: works same like find . but difference is . if any sub string not found it gives error.

```
print("i am doing nothing".index("x"))
```

```
PS D:\Python\learn_Python> python -u "d:\Python\learn_Python\exam_2.py"
Traceback (most recent call last):
  File "d:\Python\learn_Python\exam_2.py", line 1, in <module>
    print("i am doing nothing".index("x"))
    ~~~~~^~~~~~
ValueError: substring not found
```

startswith: gives Boolean value if the starting is true or false.

```
print("i am doing nothing".startswith("i am"))
```

True

endswith: gives Boolean value if the ending is true or false.

```
print("i am doing nothing".endswith("ing"))
```

True

format: it is used to fill any place letter.

```
print("my name is {} . i am {} years old".format("nabil",21))
```

my name is nabil . i am 21 years old

```
print("my name is {1} . i am {0} years old".format("nabil",21))
```

my name is 21 . i am nabil years old

we can also change position by giving position number . here nabil counted as 0th position and 21 counted as 1st position.

```
print("my name is {name} . i am {age} years old".format(name = "nabil" , age = 21))
```

my name is nabil . i am 21 years old

we can also place by giving kind of variables.

isalnum: if the the strings contains only alphabet or number is true otherwise false.

```
print("flat30".isalnum())
```

True

```
print("flat30@".isalnum())
```

False => because of @

isalpha: alphabet

```
print("flat".isalpha())
```

True

isdigit:

```
print("404".isdigit())
```

True

isidentifier: is the string follows the laws of being identifier(variable)

```
print("hello_world".isidentifier())
```

True

```
print("hello world".isidentifier())
```

False => because of space.

split: Splits every substring as string.

```
print("what is 36 july".split())
```

['what', 'is', '36', 'july']

```
print("what is 36 july".split("l"))
```

['what is 36 j', 'y']

Giving elements inside split function . we can also skip the specific part of string.

join: to join the substrings.

```
print(" ".join(['what', 'is', '36', 'july']))
```

what is 36 july

```
print("-".join(['what', 'is', '36', 'july']))
```

what-is-36-july

here we can give anything between substrings .like indicating value of sep=""

replace: we can replace specific string by this.

```
print("my name is rahim".replace("rahim", "karim"))
```

karim

strip: by using strip we can reduce extra space from string(for only one string).

```
print("      rahim      ")
```

rahim

after using strip:

```
print("      rahim      ".strip())
```

rahim